

Xebia LMS — Feature Documentation

Project: Xebia-LMS (backend artifact `LMS_backend` , package `com.geeknito.LMS_backend`)

Stack: React 19 + Vite + React Router + TanStack Query + Tailwind CSS + Framer Motion (frontend) · Spring Boot 3 + Spring Data JPA + H2/PostgreSQL + Cloudinary (backend)

Audience: New developers joining the Xebia LMS project

This document was generated by analyzing the actual source code in `frontend/src` and `backend/src/main/java/com/geeknito/LMS_backend` , together with the reference screenshot of the running admin panel. No features were invented; where the repository contains dead, duplicated, or partially-wired code this is explicitly called out so new developers don't get confused by it.

Index of Features

1. **System-Wide Notes (read this first)**
2. **Feature 1: Category Management**
3. **Feature 2: Course Management (Catalog View & Admin Course CRUD)**
4. **Feature 3: Course Builder Workspace — Modules, Submodules & Content Blocks**
5. **Feature 4: Media Library & Content Upload**
6. **Feature 5: Learning Analytics Dashboard**
7. **Feature 6: Authentication (Mock Login)**
8. **Feature 7: Application Shell — Layout, Routing, Toasts & Global State**
9. **Feature 8: Database Seeding**
10. **Feature 9: Cross-Cutting Infrastructure**
11. **Appendix A: Backend REST Endpoint Reference**
12. **Appendix B: Entity-Relationship Summary**

0. System-Wide Notes (read this first)

- **Backend base path:** no `server.servlet.context-path` is set in `application.properties` , so every `@RequestMapping` is the effective URL, e.g. `CategoryController` declares `/api/categories` → real URL is `/api/categories` . This is different from a typical setup that adds a `/api/v1` prefix — there is none here.
- **Backend port:** `server.port=${PORT:8082}` — the API listens on **8082** by default (not the common Spring default of 8080).
- **Frontend base URL:** `services/api.js` sets `baseURL = import.meta.env.VITE_API_URL || '/api'` . In dev, `vite.config.js` proxies any request under `/api` (and `/uploads`) to `http://localhost:8082` , so `api.get('/categories')` resolves to `http://localhost:8082/api/categories` — this lines up correctly with the backend's `@RequestMapping("/api/categories")` . Unlike some LMS codebases, there is **no path-prefix mismatch** here.
- **Two parallel frontend applications exist on disk.** The `frontend/` folder contains *two full copies* of the UI:
 - **ACTIVE** `frontend/src/` — a Vite + React Router SPA. This is what actually runs: `package.json` 's `"dev" / "build"` scripts both invoke `vite` , and `index.html` loads `/src/main.jsx` directly. **Every feature in this document describes this tree.**
 - **DEAD** `frontend/app/` , `frontend/components/` , `frontend/features/` , `frontend/hooks/` , `frontend/services/` , `frontend/constants/` , `frontend/utils/` , `frontend/query/` (top-level, sibling to `src/`) — an orphaned Next.js 15 App Router prototype. Its own `README.md` even documents Next.js routes and a different tech stack. Critically, **there is no next package in package.json** and no `next.config.mjs` build script wired up — `next.config.mjs` exists but nothing invokes it. This tree cannot be built or served by any script in the repo; it is inert leftover code from an earlier iteration, not a second running app. Several of its files (e.g. `features/category/CategoryManagement.jsx`) share a name with a live file under `src/` but are a materially different, disconnected implementation.

- **No authentication/authorization is actually enforced anywhere.** `AuthController.login()` checks two hardcoded credential pairs in plain Java `if` statements (`admin@xebia.com/admin123` and `instructor@xebia.com/instructor123`) and returns a fake token string (`"mock-jwt-access-token-" + System.currentTimeMillis()`) — this is not a real JWT, it is never signed, and no backend endpoint ever validates it. There is no Spring Security dependency in `pom.xml` , no filter chain, and no `@PreAuthorize` anywhere. The frontend's Axios interceptor (`services/api.js`) faithfully attaches `Authorization: Bearer <token>` and even implements a token-refresh retry flow against `/auth/refresh` — all of this is pure client-side theatre, since the backend ignores the header entirely on every other controller. `ProtectedRoute.jsx` only checks *presence* of a token in `localStorage` , so directly calling any REST endpoint with curl/Postman works with no credentials at all.
- **Deletes are hard deletes, not soft deletes — despite the UI implying otherwise.** `CategoryServiceImpl.delete()` , `CourseServiceImpl.delete()` , `ModuleServiceImpl.delete()` , `SubmoduleServiceImpl.delete()` , and `ContentServiceImpl.delete()` all call `repository.delete(entity)` , which issues a real SQL `DELETE` . The entities do carry an `isActive` boolean, but no service ever flips it to `false` on delete. The frontend, however, is built around a soft-delete mental model: `CategoryManagement.jsx` ships a **"Deleted" status filter** and a **"Restore Category" confirmation dialog** wired to a `restoreCategory()` function in `useCatalog.jsx` — but that function does nothing except show a toast ("Restore is not supported by the backend database"). In practice the "Deleted" filter can never show anything, because a hard-deleted row is gone from the API response entirely.
- **Cascade behavior contradicts the UI's delete-confirmation copy.** `CategoryManagement.jsx` 's delete dialog says "This will delete the category. Courses will remain." — but `CategoryEntity.courses` is mapped `@OneToMany(cascade = CascadeType.ALL)` , so deleting a category cascades and deletes every course in it (and, transitively, every module/submodule/content under those courses, since `CourseEntity` → `ModuleEntity` → `SubmoduleEntity` are all also `cascade = CascadeType.ALL`). The confirmation text is inaccurate.
- **Bean Validation is genuinely active** on `CourseController` , `ModuleController` , `SubmoduleController` , and `ContentController` (all use `@Valid` with DTOs that carry real `@NotBlank` / `@Size` / `@NotNull` annotations), and `GlobalExceptionHandler` correctly turns validation failures into a 400 with per-field messages. `CategoryController` also uses `@Valid` , and `CategoryRequestDTO` requires `name` .
- **Employees / Enrollments / Certifications / Fresher Journeys are analytics-only.** These four entities (and their repositories) exist solely to feed `AnalyticsController` 's dashboard endpoint. There are no CRUD controllers, DTOs, or admin screens for them anywhere — they are only ever created once, in bulk, by the startup database seeder (see Feature 8), and only ever read back in aggregate by Feature 5.
- **App is a client-side routed SPA** using `react-router-dom` 's `BrowserRouter` (not a state-machine like some LMS prototypes) — real URLs like `/admin/courses/12/edit` exist and are bookmarkable/shareable, and are guarded per-route by `ProtectedRoute` .

Feature 1: Category Management

1. Feature Name

Category Management (Course Taxonomy)

2. Feature Purpose

Lets an LMS administrator create, list, edit, and delete the top-level categories that courses are grouped under (e.g. "Cloud Computing", "Artificial Intelligence" — as seeded and shown in the reference screenshot). Categories carry a display icon/emoji, a brand color, and optional logo/banner/background/thumbnail image URLs, and drive the color-coded badges and filter dropdowns used throughout the Course Management screen. There is no learner-facing role in this codebase; every screen is an admin console.

3. Feature Location

Frontend

- List/grid page: `frontend/src/features/category/CategoryManagement.jsx` (routes: `/admin/categories` , `/admin/categories/:categoryId` via `CategoryCoursesPage.jsx`)
- Create/edit form: `frontend/src/features/category/CategoryForm.jsx` (routes: `/admin/categories/new` , `/admin/categories/:categoryId/edit`)
- Cards/rows: `frontend/src/components/catalog/CategoryCard.jsx` (exports both `CategoryCard` grid tile and `CategoryRow` table row)
- Global state + API calls: `frontend/src/hooks/useCatalog.jsx`
-  **Dead** duplicate: `frontend/features/category/CategoryManagement.jsx` (orphaned Next.js tree, see System-Wide Notes)

Backend

- Controller: `controller/CategoryController.java` → `/api/categories`
- Service interface/impl: `service/CategoryService.java` / `serviceImpl/CategoryServiceImpl.java`
- Repository: `repository/CategoryRepository.java`
- Entity: `entity/learning/CategoryEntity.java`
- DTOs: `dto/CategoryRequestDTO.java` , `dto/CategoryResponseDTO.java` , `dto/CategoryWiseCourseResponseDTO.java`
- Mapper: `mapper/CategoryMapper.java` (plain static-method mapper, not MapStruct despite the `mapper` package name)

4. Related Files

Frontend: `CategoryManagement.jsx` · `CategoryForm.jsx` · `CategoryCard.jsx` · `hooks/useCatalog.jsx` · `services/api.js` · `components/ui/Modal.jsx` (`ConfirmationDialog`) · `components/ui/StatCard.jsx` · `components/ui/Pagination.jsx`

Backend: `CategoryController.java` · `CategoryServiceImpl.java` · `CategoryRepository.java` · `CategoryEntity.java` · `CategoryRequestDTO.java` / `CategoryResponseDTO.java` · `CategoryMapper.java` · `exception/GlobalExceptionHandler.java`

5. Feature Entry Point

Admin clicks "Create Category" on `/admin/categories` → navigates to `/admin/categories/new` (`CategoryForm.jsx`) → fills name/description/icon/color/images → submit → `useCatalog.createCategory(payload)` → `mapCategoryToBackendPayload()` → Axios `POST /api/categories` → `CategoryController.createCategory()` → `CategoryServiceImpl.create()` → `CategoryRepository.save()` → PostgreSQL/H2 `categories` table.

6. Complete Feature Lifecycle (Create flow)

User fills `CategoryForm.jsx` (name, description, icon picker/upload, color swatch, banner/logo/thumbnail image uploader) → `handleSubmit()` builds a local form object → `createCategory(payload)` in `useCatalog.jsx` → `mapCategoryToBackendPayload()` reshapes it to `{ name, description, icon, color, isActive, logo, bannerImage, backgroundImage, thumbnail }` → `api.post('/categories', backendPayload)` → `CategoryController.createCategory(@Valid @RequestBody CategoryRequestDTO)` → `CategoryServiceImpl.create()` builds a `CategoryEntity` via `CategoryMapper.toEntity()` → `CategoryRepository.save()` issues `INSERT INTO categories` → response wrapped in `ApiResponse("Category created successfully", CategoryResponseDTO)` , HTTP 201 → back in the frontend, `createCategory()` calls `await refreshData()` , which re-fetches *both* `/categories` and `/courses` in parallel and recomputes `courseCount` client-side by filtering courses per category → a success toast fires → form navigates back to the list, which now shows the new card.

7. API Details

Method	Endpoint	Request Body	Response	Auth	Notes
GET	<code>/api/categories</code>	—	<code>ApiResponse<CategoryResponseDTO[]></code>	None	Returns a not populated, computes
GET	<code>/api/categories/{id}</code>	—	<code>ApiResponse<CategoryResponseDTO></code>	None	404 via Resource missing
GET	<code>/api/categories/{categoryId}/courses</code>	—	<code>ApiResponse<CategoryWiseCourseResponseDTO></code>	None	Not called from the frontend though it is on the backend
POST	<code>/api/categories</code>	<code>CategoryRequestDTO</code> (name required)	<code>ApiResponse<CategoryResponseDTO></code> (201)	None	
PUT	<code>/api/categories/{id}</code>	<code>CategoryRequestDTO</code>	<code>ApiResponse<CategoryResponseDTO></code>	None	Full field-update
DELETE	<code>/api/categories/{id}</code>	—	<code>ApiResponse</code> (200, data:null)	None	Hard delete of courses/notes in that category (Notes)

Error responses: 404 (`ResourceNotFoundException`), 400 (`MethodArgumentNotValidException` — missing `name`), 500 generic fallback.

8. Database Flow

Table: `categories` (`id` , `name` `UNIQUE NOT NULL`, `icon` , `description` `TEXT`, `color` , `is_active` , `logo` , `banner_image` , `background_image` , `thumbnail` , `created_at` , `updated_at`). One-to-many with `courses` (`category_id` `FK`, **`NOT NULL`**, `cascade ALL`). `CategoryRepository` only exposes one custom query — `findByIdWithCourses()` (an `@EntityGraph` fetch) — which is used solely by the "get category's courses" endpoint that the frontend never calls. All list/create/update/delete flows use plain `JpaRepository` methods (`findAll` , `save` , `delete`).

9. State Management

All catalog data (`categories` + `courses`) lives in one React Context, `CatalogProvider` in `useCatalog.jsx` — there is no TanStack Query cache in play for this data even though `@tanstack/react-query` is a dependency and `QueryClientProvider` wraps the app in `providers.jsx`; nothing in the active category/course code path actually calls `useQuery` / `useMutation`. Instead, every mutation (`createCategory` , `updateCategory` , `deleteCategory`) manually calls `await refreshData()` afterward, which re-fetches the full category and course lists from scratch — a "refetch everything" strategy rather than cache invalidation or optimistic updates. Local UI state (search text, filters, pagination, sort, view mode, delete-confirmation target) lives in page-local `useState` inside `CategoryManagement.jsx`.

10. Component Hierarchy

```
CategoryManagement (page)
├─ StatCard ×4 (Total / Active / Inactive / Total Courses)
├─ grid: CategoryCard ×N | table: CategoryRow ×N
├─ Pagination
└─ ConfirmationDialog (delete / "restore")
```

11. Sequence Diagram (Text)

Admin → CategoryManagement.jsx → useCatalog (createCategory) → api.js (Axios) → CategoryController → CategoryServiceImpl → CategoryRepository → DB → CategoryRepository → CategoryServiceImpl → CategoryController → api.js → useCatalog.refreshData() → GET /categories + GET /courses (parallel) → CatalogContext re-render → CategoryManagement.jsx

12. Dependencies

Frontend: `axios` , `lucide-react` , `framer-motion` , `react-router-dom` . Backend: Spring Web, Spring Data JPA, Spring Validation (`jakarta.validation`), Lombok.

13. Validation Rules

Frontend: name required before submit (client-side guard in `CategoryForm.jsx`). Backend: `@Valid` is active and enforced — `name` is `@NotBlank` and capped at 100 chars; `icon` capped at 1000 chars; `color` capped at 20 chars. The DB-level `UNIQUE` constraint on `name` has **no dedicated exception handler** — a duplicate name throws a raw `DataIntegrityViolationException` , which falls through to `GlobalExceptionHandler` 's catch-all `Exception` handler and surfaces as a generic 500, not a clean 409.

14. Error Handling

Frontend: every mutation is wrapped in try/catch inside `useCatalog.jsx` ; on failure it shows `error.response?.data?.message || 'Failed to <action> category'` via the toast system and re-throws so the calling form can also react. Backend: `GlobalExceptionHandler` maps `ResourceNotFoundException` → 404, `MethodArgumentNotValidException` → 400 with field-level messages, `IllegalArgumentException` → 400, anything else → 500 with stack trace printed to console.

15. Security

None. See System-Wide Notes. CORS is handled once, globally, by `config/CorsConfig.java` (allows any `localhost / 127.0.0.1` origin on any port, credentials enabled) — no per-controller `@CrossOrigin` overrides exist anywhere in this codebase.

16. Important Classes and Methods

`CategoryController.createCategory/getAllCategories/getCategoryById/getCategoryCourses/updateCategory/deleteCategory` — REST entry points.

`CategoryServiceImpl.create/update/delete` — thin pass-through to the repository via the mapper.

`CategoryMapper.toEntity/toResponseDTO/updateEntity/toCategoryWiseCourseResponseDTO` — manual (non-MapStruct) conversion.

`useCatalog.mapBackendCategory()` / `mapCategoryToBackendPayload()` — frontend ↔ backend shape translation, including deriving placeholder logo/banner/thumbnail images via `getAIPlaceholderImage()` when the backend value is empty.

17. Feature Dependencies

Course Management (category dropdown when creating/editing a course; category badge/color on course cards). No auth dependency.

18. Possible Improvements

- Either implement real soft-delete (flip `isActive`) to match the frontend's "Deleted"/"Restore" UI, or remove that UI since it is currently non-functional.
- Add a `DataIntegrityViolationException` handler for a clean 409 on duplicate category names.
- Fix the delete-confirmation copy ("Courses will remain") to reflect the actual cascade-delete behavior, or change the cascade to `SetNull` /no-cascade if the stated intent is correct.
- Either wire up TanStack Query for real caching/invalidation, or remove the unused `QueryClientProvider` wrapper to reduce confusion.
- Delete or clearly quarantine the orphaned Next.js prototype tree at the frontend root.

19. Summary

Admin-only CRUD over `categories`, no auth enforced, Bean Validation genuinely active on create/update. Active path:

`CategoryManagement.jsx` / `CategoryForm.jsx` → `useCatalog.jsx` → `api.js` → `CategoryController` → `CategoryServiceImpl` → `CategoryRepository`. Deletes are real, cascading hard deletes despite frontend copy and UI (status filter, restore dialog) that assume soft-delete semantics — this is the single most important gotcha in this feature.

[↑ Back to top / index](#)

Feature 2: Course Management (Catalog View & Admin Course CRUD)

1. Feature Name

Course Management — All Courses catalog grid/table plus the two-step Create/Edit Course wizard

2. Feature Purpose

The primary "home" screen of the admin panel (this is the screen captured in the reference screenshot: "All Courses", search + Level/Category/Status filters, grid/list toggle, "2 Courses / 2 Active / 2 Published" pill counters, purple sidebar). Lets an admin browse, search, filter, sort, and paginate every course, then create a new one or edit an existing one through a 2-step form (Basic Details → SEO & Meta), and duplicate or delete a course from the grid.

3. Feature Location

Frontend

- Catalog grid/table: `frontend/src/features/course/CourseManagement.jsx`, mounted by `pages/AllCoursesPage.jsx` (route `/admin/courses`) and, scoped to one category, by `pages/CategoryCoursesPage.jsx` (route `/admin/categories/:categoryId`)
- Create/edit wizard: `frontend/src/features/course/CourseForm.jsx`, mounted by `pages/CourseFormPage.jsx` (routes `/admin/courses/new`, `/admin/courses/:courseId/edit`)
- Cards/rows: `frontend/src/components/catalog/CourseCard.jsx` (exports `CourseCard` and `CourseRow`)
- Global state + API calls: `frontend/src/hooks/useCatalog.jsx`
-  **Dead** duplicates in the orphaned Next.js tree: `frontend/features/course/CourseManagement.jsx`

Backend

- Controller: `controller/CourseController.java` → `/api/courses`
- Service interface/impl: `service/CourseService.java` / `serviceImpl/CourseServiceImpl.java`
- Repository: `repository/CourseRepository.java`
- Entity: `entity/learning/CourseEntity.java` — by far the largest entity in the schema (60+ columns, mostly SEO/programmatic-SEO fields: meta tags, Open Graph, Twitter Card, JSON-LD schema markup, FAQ content, custom head/body scripts, semantic keywords, etc.)
- DTOs: `dto/CourseRequestDTO.java` , `dto/CourseResponseDTO.java`
- Mapper: `mapper/CourseMapper.java` (manual static mapper)
- Media upload used by the image pickers: `controller/MediaUploadController.java` → `/api/media/upload` (see Feature 4)

4. Related Files

Frontend: `CourseManagement.jsx` · `CourseForm.jsx` · `CourseCard.jsx` · `hooks/useCatalog.jsx` · `constants/index.js`
(`DIFFICULTY_LEVELS` , `LANGUAGES` , `TECHNOLOGIES` , `COURSE_STATUSES`) · `utils/index.js` (`slugify` , `paginate`) ·
`components/ui/Pagination.jsx` · `components/ui/Modal.jsx`
Backend: `CourseController.java` · `CourseServiceImpl.java` · `CourseRepository.java` · `CourseEntity.java` ·
`CourseRequestDTO.java` / `CourseResponseDTO.java` · `CourseMapper.java` · `CategoryRepository.java` (category lookup on create/update)

5. Feature Entry Point

Admin lands on `/admin/courses` (default post-login redirect for the "Courses" nav item) → sees the grid/table populated from the `courses` array already held in `CatalogContext` (fetched on app hydration, see Feature 7) → clicks "Create Course" → if no categories exist yet, a toast blocks navigation ("Create a category first before adding a course.") → otherwise navigates to `/admin/courses/new` → `CourseForm.jsx` step 1.

6. Complete Feature Lifecycle (Create flow)

Step 1 — Basic Details: title, category (dropdown sourced from `useCatalog().categories`), technology (from a ~25-item hardcoded `TECHNOLOGIES` map used only for a colored badge), difficulty (`Beginner|Intermediate|Advanced|Expert`), duration, language, short/long description, logo/banner/thumbnail image (uploaded straight to Cloudinary via `POST /media/upload` , see Feature 4), YouTube/preview video URLs, status (`draft|in_review|published|archived` in the UI constant, but the backend only actually persists two independent booleans — see note below). The slug auto-generates from the title via `slugify()` but is editable.

Step 2 — SEO & Meta: meta title/description/keywords, canonical URL, primary/secondary/focus keywords, robots directive, Open Graph fields, Twitter Card fields, raw JSON-LD schema/FAQ/breadcrumb markup textareas, learning outcomes/prerequisites/target audience (each stored as a JSON-stringified array in a TEXT column), course highlights, career opportunities, a numeric SEO score, and indexing/search-visibility toggles.

Submit → `mapCourseToBackendPayload()` flattens the 4-state UI status down to the two real backend booleans: `status === 'published' ⇒ isPublished:true` ; `status === 'archived' ⇒ isActive:false` ; anything else (including `draft` and `in_review`) leaves `isActive:true` , `isPublished:false` — meaning **"Draft" and "In Review" are visually distinct options in the form but are functionally identical and indistinguishable once saved**, since the backend has no field for "in review". → `api.post('/courses', payload)` → `CourseController.createCourse(@Valid CourseRequestDTO)` → `CourseServiceImpl.create()` looks up the parent `CategoryEntity` (404 via `ResourceNotFoundException` if the category ID doesn't exist) → `CourseMapper.toEntity()` → `CourseRepository.save()` → 201 response → frontend calls `refreshData()` and shows a success toast, then navigates back to the list.

7. API Details

Method	Endpoint	Request Body	Response	Auth	Notes
GET	<code>/api/courses</code>	—	<code>ApiResponse<CourseResponseDTO[]></code>	None	Includes nested <code>modules / submodules / contents</code> when present (see §16) — this is what the reference screenshot's "1 Modules · 1 Submodules" / "0 Modules · 0 Submodules" counters read from
GET	<code>/api/courses/{id}</code>	—	<code>ApiResponse<CourseResponseDTO></code>	None	Full curriculum tree, used by the Course Builder Workspace

POST	/api/courses	CourseRequestDTO (title , slug , categoryId required)	ApiResponse<CourseResponseDTO> (201)	None	404 if categoryId doesn't exist
PUT	/api/courses/{id}	CourseRequestDTO	ApiResponse<CourseResponseDTO>	None	Full field overwrite (re-resolves category each time)
DELETE	/api/courses/{id}	—	ApiResponse (200)	None	Hard delete , cascades to modules/submodules/content

8. Database Flow

Table: `courses` (60+ columns — see Entity list above), indexed on `slug`, `primaryKeyword`, `isActive`, `isFeatured`, `level`, `title`. `slug` is `UNIQUE`, `NOT NULL`. Many-to-one to `categories` (`category_id` FK, **NOT NULL**). One-to-many to `modules` (cascade ALL). `CourseRepository` exposes `findBySlug()` (unused by any controller today — a public slug-lookup endpoint was apparently planned but never built), `findByIdWithModules()` (`@EntityGraph`, also unused — `getById` instead relies on manually walking the lazy collections inside the read-only transaction to force-initialize them), and `findAllWithCategory()` (also unused — same story, `getAll()` touches `course.getCategory().getName()` per row instead).

9. State Management

Same `CatalogContext` pattern as Category Management (see Feature 1, §9) — no TanStack Query, "refetch everything" after each mutation. Local page state in `CourseManagement.jsx`: search, level/category/status filters, sort key, page/page size, card-vs-list view toggle, delete-confirmation target. The 2-step wizard's step index and the entire form object live in local `useState` inside `CourseForm.jsx` (1159 lines) — there is no draft-autosave or step-level validation persistence; navigating away loses all progress.

10. Component Hierarchy

- `CourseManagement` (page, optionally scoped by `categoryId` prop)
 - filter toolbar (search, Level select, Category select, Status select, grid/list toggle)
 - grid: `CourseCard` ×N | table: `CourseRow` ×N (columns match the reference screenshot: #, Course Name/Slug, Category, Difficulty, Curriculum Stats, Language/Duration, Last Updated, Status, Featured, Actions)
 - Pagination
 - `ConfirmationDialog` (delete)
- `CourseForm` (page)
 - Step 1 panel: image uploader ×3, title/category/technology/difficulty/duration/language selects, description textareas
 - Step 2 panel: SEO/meta/OG/Twitter/schema field groups

11. Sequence Diagram (Text)

Admin → `CourseForm.jsx` (step 1 → step 2 → submit) → `useCatalog.createCourse()` → `mapCourseToBackendPayload()` → `api.js` (Axios) → `CourseController` → `CourseServiceImpl` (resolves `CategoryEntity`) → `CourseMapper.toEntity()` → `CourseRepository.save()` → DB → response bubbles back → `refreshData()` (GET /categories + GET /courses) → `CatalogContext` → `CourseManagement.jsx` re-render

12. Dependencies

Frontend: `axios`, `lucide-react`, `react-router-dom`. Backend: Spring Web, Spring Data JPA, Spring Validation, Lombok. Depends on Category Management (category must exist first) and Media Upload (image fields).

13. Validation Rules

Frontend: `title` and `categoryId` required before Step 1 → Step 2 transition is allowed. Backend (genuinely enforced via `@Valid`): `title @NotBlank /max 200`, `slug @NotBlank /max 250` (and `UNIQUE` at the DB level, with the same unhandled- `DataIntegrityViolationException` - falls-to-500 gap as `Category`), `level` / `language` / `duration` / `icon` / `thumbnail` / `bannerImage` all have `@Size` caps but are optional.

14. Error Handling

Same pattern as Category Management: try/catch in `useCatalog.jsx` mutation functions, toast on failure, `GlobalExceptionHandler` on the backend. One extra case: `handleCreate()` in `CourseManagement.jsx` pre-emptively blocks navigation to the create form with a toast if `categories.length === 0`, avoiding a guaranteed 404-on-submit.

15. Security

None (see System-Wide Notes). Same global CORS policy as every other controller.

16. Important Classes and Methods

`CourseController.createCourse/getAllCourses/getCourseById/updateCourse/deleteCourse` .
`CourseServiceImpl.getAll()` — iterates all courses and, inside the read-only transaction, touches `course.getCategory().getName()` per row purely to force-initialize the lazy category association before the transaction closes (a manual alternative to using the unused `findAllWithCategory()` query).
`CourseServiceImpl.getById()` — similarly walks `modules` → `submodules` → `contents` to force-initialize the whole lazy tree before mapping.
`CourseMapper.toResponseDTO()` — maps ~60 fields and also maps nested `modules` whenever the lazy collection is already initialized; `toResponseDTOWithModules()` is a same-signature alias that just calls `toResponseDTO()` — the "with/without modules" distinction implied by the two method names does not actually exist in the current implementation.
`useCatalog.mapBackendCourse()` / `mapCourseToBackendPayload()` — translates the flat backend DTO into/out of the richer frontend course shape, including the `technology` ⇌ `seoCategory` and `difficulty` ⇌ `level` field renames, and the 4-state-UI-to-2-boolean-backend status squashing described in §6.
`duplicateCourse()` in `useCatalog.jsx` — client-orchestrated deep copy: creates the course, then loops through its `modules` → `submodules` → `contents`, issuing a separate `POST` for each, sequentially, awaiting each response before issuing the next (no batch/bulk endpoint exists on the backend).

17. Feature Dependencies

Category Management (parent taxonomy), Media Upload (all image fields, see Feature 4), Course Builder Workspace (curriculum authored after the course itself is created).

18. Possible Improvements

- Give "Draft" and "In Review" real backend representation (e.g. a proper `status` enum column) instead of silently collapsing both into the same `isActive:true, isPublished:false` state.
- Add a `DataIntegrityViolationException` handler for duplicate slugs (currently a 500).
- Either use the existing `findBySlug()` / `findAllWithCategory()` / `findByIdWithModules()` queries or delete them — they're currently unused, hand-rolled equivalents exist in the service layer instead.
- Rename or collapse `toResponseDTOWithModules()` since it no longer differs from `toResponseDTO()` .
- Add a bulk/batch duplicate endpoint to avoid N sequential round-trips when copying a course with many modules.

19. Summary

The core catalog screen and 2-step course wizard. Active path: `CourseManagement.jsx` / `CourseForm.jsx` → `useCatalog.jsx` → `api.js` → `CourseController` → `CourseServiceImpl` → `CourseMapper` → `CourseRepository` . The UI's 4-state course status is a leaky abstraction over a 2-boolean backend model. Hard delete + cascade apply here too. `CourseEntity` carries an unusually large, mostly-unused-by-the-UI SEO/programmatic-SEO surface area (schema markup, FAQ content, custom scripts) that Step 2 of the wizard exposes as raw textareas.

[↑ Back to top / index](#)

Feature 3: Course Builder Workspace — Modules, Submodules & Content Blocks

1. Feature Name

Course Builder Workspace (Curriculum Authoring)

2. Feature Purpose

Once a course exists, this is where its curriculum is authored: Modules → Submodules → Content Blocks, three levels deep, matching the `CourseEntity → ModuleEntity → SubmoduleEntity → ContentEntity` chain. This is the largest single file in the frontend (`CourseBuilderWorkspace.jsx`, ~1,985 lines) — it is a self-contained, monolithic workspace component that inlines its own module/submodule list panels, its own content-block inline editor, and its own drag-and-drop reordering, rather than composing the smaller "builder" components that also exist on disk.

3. Feature Location

Frontend

- Main workspace: `frontend/src/features/course/CourseBuilderWorkspace.jsx`, mounted by `pages/CourseBuilderPage.jsx` (route `/admin/curriculum/:courseId`)
- Empty-state landing: `frontend/src/features/curriculum/CurriculumLanding.jsx` (route `/admin/curriculum`, reached only if a user manually types the URL with no `:courseId` — there is no sidebar link to it)
- Global state + API calls: `frontend/src/hooks/useCatalog.jsx` (module/submodule/content CRUD functions)
- ⚠ **Unused sibling components** in `frontend/src/components/builder/` — none of these are imported by `CourseBuilderWorkspace.jsx` or any other live file:
 - `CourseHierarchyTree.jsx` and `HierarchyContextMenu.jsx` — a more polished tree-view + right-click-menu pattern that was apparently designed to replace the workspace's inline panel-based navigation, but was never wired in
 - `CourseDashboardCards.jsx` — stat-card summary for a single course, not rendered anywhere
 - `CoursePublishDialog.jsx` — a dedicated publish/unpublish confirmation dialog, not rendered anywhere (publishing instead happens implicitly through the course status field in `CourseForm.jsx`)

Only `ContentPreviewDrawer.jsx` from this folder is actually used, and only by `MediaLibrary.jsx` (Feature 4), not by the builder itself.

- ⚠ **Unused alternative feature files** — `frontend/src/features/module/ModuleManager.jsx` and `frontend/src/features/content/ContentManager.jsx` exist and appear to implement standalone module/content CRUD screens, but neither is imported by `App.jsx` or any page — they are unreachable dead code superseded by the inline editors inside `CourseBuilderWorkspace.jsx`.

Backend

- Modules: `controller/ModuleController.java` → `/api/modules`, `service(Impl)/ModuleService(Impl).java`, `repository/ModuleRepository.java`, `entity/learning/ModuleEntity.java`, `dto/ModuleRequestDTO.java` / `ModuleResponseDTO.java`, `mapper/ModuleMapper.java`
- Submodules: `controller/SubmoduleController.java` → `/api/submodules`, `service(Impl)/SubmoduleService(Impl).java`, `repository/SubmoduleRepository.java`, `entity/learning/SubmoduleEntity.java`, `dto/SubmoduleRequestDTO.java` / `SubmoduleResponseDTO.java`, `mapper/SubmoduleMapper.java`
- Content: `controller/ContentController.java` → `/api/contents`, `service(Impl)/ContentService(Impl).java`, `repository/ContentRepository.java`, `entity/learning/ContentEntity.java`, `dto/ContentRequestDTO.java` / `ContentResponseDTO.java`, `mapper/ContentMapper.java`

4. Related Files

Frontend: `CourseBuilderWorkspace.jsx` · `hooks/useCatalog.jsx` · `services/api.js` · `constants/index.js` (`CONTENT_TYPES`) · `components/ui/Modal.jsx`

Backend: `ModuleController/Service/Repository/Entity/DTO/Mapper` · `SubmoduleController/Service/Repository/Entity/DTO/Mapper` · `ContentController/Service/Repository/Entity/DTO/Mapper`

5. Feature Entry Point

From the course grid/table (Feature 2), an admin clicks a course row/card's builder action → navigates to `/admin/curriculum/:courseId` → `CourseBuilderPage.jsx` resolves the course from the already-hydrated `CatalogContext` via `getCourse(courseId)` (a "Course Not Found" panel renders if the course is missing/deleted) → mounts `CourseBuilderWorkspace`, defaulting `activeModuleId` to the course's first module (if any).

6. Complete Feature Lifecycle

The workspace has two "views" toggled by local state (`activeView`): `modules_submodules` (browse/manage the module and submodule lists side by side) and `submodule_content` (drill into one submodule's ordered content blocks).

Add Module: inline form opens (`moduleFormOpen='add'`) → `title/description/order` → `addModule(courseId, payload)` in `useCatalog.jsx` computes the next `moduleOrder` as `(existingModules.length + 1)` → `mapModuleToBackendPayload()` → `POST /api/modules` → `ModuleController.createModule()` → `ModuleServiceImpl.create()` resolves the parent `CourseEntity` → `ModuleRepository.save()` → `refreshData()` re-fetches the whole catalog.

Add Submodule: same pattern, nested one level deeper — `addSubmodule(courseId, moduleId, payload)` auto-generates a slug via `slugify()` if one isn't supplied, appending a short random suffix so submodule slugs stay globally unique across the whole database (submodule slug has a global `UNIQUE` constraint, not scoped per-module).

Add Content Block: the deepest level — `addContent(courseId, moduleId, submoduleId, payload)` → `mapContentToBackendPayload()` has a notable dual-encoding scheme (see §16) → `POST /api/contents` .

Reordering: `reorderModules/reorderSubmodules/reorderContent` each take an ordered array of IDs (driven by drag-and-drop) and fire one `PUT` per item in parallel via `Promise.all` , each carrying a freshly computed 1-based `order` field — there is no dedicated bulk-reorder endpoint on the backend.

Duplicate (at any level): `duplicateModule/duplicateSubmodule/duplicateContent` recursively re-create everything beneath the duplicated node with new IDs, sequentially awaiting each nested `POST` — the same N-sequential-requests pattern as `duplicateCourse` (Feature 2).

7. API Details

Resource	Method	Endpoint	Notes
Module	GET	<code>/api/modules</code>	Flat list with parent <code>course</code> summary embedded; not used by the workspace (which reads modules off the already-loaded course tree instead)
Module	GET	<code>/api/modules/{id}</code>	Returns nested <code>submodules</code>
Module	POST / PUT / DELETE	<code>/api/modules /</code> <code>/api/modules/{id}</code>	<code>title</code> required; DELETE is hard delete, cascades to submodules/content
Submodule	GET	<code>/api/submodules ,</code> <code>/api/submodules/{id}</code>	Same flat-list-unused / by-id-nested pattern as Module
Submodule	POST / PUT / DELETE	<code>/api/submodules /</code> <code>/api/submodules/{id}</code>	<code>title , slug , moduleId</code> required; slug globally <code>UNIQUE</code>
Content	GET	<code>/api/contents ,</code> <code>/api/contents/{id}</code>	List endpoint eagerly fetches parent submodule via <code>findAllWithSubmodule()</code>
Content	POST / PUT / DELETE	<code>/api/contents /</code> <code>/api/contents/{id}</code>	<code>type , submoduleId</code> required

8. Database Flow

`modules` (FK `course_id` NOT NULL, cascade ALL from course) → `submodules` (FK `module_id` NOT NULL, cascade ALL from module, `slug` `UNIQUE`) → `contents` (FK `submodule_id` NOT NULL, cascade ALL from submodule). All three carry an `*_order` integer column (`moduleOrder / submoduleOrder / contentOrder`) that the frontend maintains client-side during drag-and-drop and persists via the sequential- `PUT` `reorder` functions described above. `ModuleRepository / SubmoduleRepository` each define an `@EntityGraph` "with children" query (`findByIdWithSubmodules , findByIdWithContents`) that *are* actually used by their respective `getById()` service methods — unlike the equivalent unused queries on `CourseRepository` .

9. State Management

All server truth still flows through the single `CatalogContext` (Feature 1 §9) — the workspace calls `getCourse(courseId)` to read the current tree and calls the various `add*/update*/delete*/duplicate*/reorder*` functions to mutate it, every one of which ends in a full `refreshData()` . Locally, `CourseBuilderWorkspace.jsx` owns a large amount of its own `useState` : which module/submodule is "active" (drilled into), which inline form is open (`moduleFormOpen / submoduleFormOpen / contentFormOpen` , each one of `'add'|'edit'|null`), the full content-block editor form object, and upload-in-progress flags for the content editor's own file picker.

10. Component Hierarchy

CourseBuilderPage

└─ Breadcrumb

└─ CourseBuilderWorkspace (monolithic — module list panel, submodule list panel, and content-block editor are all inlined in this one file rather than split into the sibling components in `components/builder/`)

└─ ConfirmationDialog (delete at any level)

11. Sequence Diagram (Text)

Admin → CourseBuilderWorkspace.jsx (add/edit/delete/reorder/duplicate at Module, Submodule, or Content level) → useCatalog.jsx (matching function) → api.js (Axios) → Module/Submodule/ContentController → matching ServiceImpl (resolves parent entity) → matching Mapper → matching Repository.save()/delete() → DB → refreshData() (GET /categories + GET /courses, full tree) → CatalogContext → CourseBuilderWorkspace.jsx re-render

12. Dependencies

Frontend: `axios`, `framer-motion` (drag animations), `lucide-react`. Backend: Spring Web, Spring Data JPA, Spring Validation, Lombok. Hard dependency on Course Management (a course must exist first).

13. Validation Rules

Backend, all genuinely enforced via `@Valid : Module title @NotBlank /max 200, courseId @NotNull . Submodule title @NotBlank /max 200, slug @NotBlank /max 250, moduleId @NotNull`, plus `@Size` caps on the SEO/OG fields it also carries (`metaTitle max 70, metaDescription max 320`, etc. — submodules have their own miniature SEO surface, separate from the course's). Content `type @NotBlank /max 30, submoduleId @NotNull` — note **type is a free-text string with no enum/whitelist validation server-side**; the frontend's `CONTENT_TYPES` constant (`video, pdf, ppt, doc, notes, image, link`) is a client-side convention only, not enforced by the API.

14. Error Handling

Same try/catch-plus-toast pattern in every `useCatalog.jsx` mutation, same `GlobalExceptionHandler` backend mapping. Because reordering and deep-duplicate operations fire multiple sequential/parallel requests, a failure partway through (e.g. network drop after duplicating 2 of 5 submodules) leaves the database in a **partially-duplicated state** with no rollback — there is no backend transaction spanning the whole client-orchestrated operation, since each HTTP call is its own independent `@Transactional` service method.

15. Security

None (see System-Wide Notes).

16. Important Classes and Methods

`useCatalog.mapContentToBackendPayload()` — the most interesting piece of logic in this feature: for "structured" content types (`text, notes, code, heading, callout, table`) the block's Markdown body is written straight into `ContentEntity.text`; for every other type (`video, pdf, ppt, doc, image, link`, etc.) the `text` column is instead repurposed to store a **JSON-stringified metadata blob** (`{ fileSize, fileSize, duration, pageCount, slideCount }`), because the entity has no dedicated columns for those fields. `mapBackendContent()` reverses this by attempting `JSON.parse(ct.text)` for non-Markdown types and silently falling back to empty metadata if parsing fails.

`ModuleServiceImpl / SubmoduleServiceImpl / ContentServiceImpl` — all three follow an identical shape: resolve the parent entity by ID (404 if missing) → map DTO→entity via the corresponding static `Mapper` → save.

`CourseBuilderWorkspace.jsx` 's internal `contentForm` state — a single shared form object reused for every content type, with type-specific fields (heading level, code language, alt/caption) all present simultaneously regardless of which type is selected.

17. Feature Dependencies

Course Management (parent course must exist). Media Library (content blocks of file-based types link to files uploaded through the same Cloudinary pipeline as course images, see Feature 4).

18. Possible Improvements

- Delete the four unreachable dead files (`ModuleManager.jsx`, `ContentManager.jsx`, and the unused `components/builder/` siblings) or finish wiring them in and retire the monolithic inline implementation.

- Give `ContentEntity` proper typed columns for file metadata instead of overloading the `text` column with conditionally-JSON-encoded data.
- Add server-side validation/whitelisting of the `type` field.
- Add bulk reorder and bulk duplicate endpoints to remove the N-sequential-request pattern and its partial-failure risk.

19. Summary

A monolithic, single-file curriculum authoring workspace covering Module → Submodule → Content, each level following the identical create/read/update/delete/reorder/duplicate pattern against its own REST resource and its own manual mapper. Several purpose-built sibling components and two entire alternative feature screens exist on disk but are never imported — dead code left over from earlier refactors. Content blocks use a dual-purpose `text` column (raw Markdown vs. JSON-encoded file metadata) depending on content type, which is the feature's most important implementation detail for anyone extending it.

[↑ Back to top / index](#)

Feature 4: Media Library & Content Upload

1. Feature Name

Media Library (asset browser) + Upload Content page (standalone content-block creation flow)

2. Feature Purpose

Gives the admin a single searchable/filterable grid of every non-text media asset attached to any course's content blocks, and a dedicated full-page flow for uploading a new file and attaching it to a course/module/submodule as a new content block, without having to open the Course Builder Workspace.

3. Feature Location

Frontend

- Library grid: `frontend/src/features/content/MediaLibrary.jsx` (route `/admin/media`)
- Upload flow: `frontend/src/pages/UploadContentPage.jsx` (route `/admin/upload-content`, reached only via a button inside `MediaLibrary.jsx` — there is no sidebar nav entry for either page, see Feature 7)
- Preview: `frontend/src/components/builder/ContentPreviewDrawer.jsx`

Backend

- `controller/MediaUploadController.java` → `/api/media/upload`
- `service/CloudinaryService.java` / `serviceImpl/CloudinaryServiceImpl.java`
- Content persistence reuses `ContentController` / `ContentServiceImpl` from Feature 3

4. Related Files

Frontend: `MediaLibrary.jsx` · `UploadContentPage.jsx` · `hooks/useCatalog.jsx` (`mediaLibrary` derived list, `addContent`) · `components/ui/ImageUploader.jsx` · `components/ui/RichNotesEditor.jsx` · `utils/placeholderUtils.js`

Backend: `MediaUploadController.java` · `CloudinaryServiceImpl.java`

5. Feature Entry Point

Admin navigates directly to `/admin/media` (no sidebar link — must be reached via a deep link, or via the "Upload Content" button chain that starts elsewhere) → sees a computed grid of every file-type content block across every course → clicks "Upload Content" → `/admin/upload-content` → picks a category, then a course, then a module, then a submodule (four cascading dropdowns, each filtered by the previous selection using local component state, not by any dedicated backend query) → picks a content type and either drags a file in or fills the appropriate fields → submits.

6. Complete Feature Lifecycle

Media Library is not a real backend resource — there is no `/api/media` list endpoint and no `media` database table. `useCatalog.jsx`'s `mediaLibrary` value is a `useMemo`-derived array computed entirely client-side by walking every course → module → submodule → content in the already-loaded `CatalogContext` tree and flattening every content block whose `type` is not `link` or `notes` into a flat list with a synthetic composite ID (`${courseId}-${moduleId}-${submoduleId}-${contentId}`). This means the Media Library's contents, filters, and counts are always a byproduct of the Course Builder data, recomputed on every catalog refresh.

Upload flow: file selected → `UploadContentPage.jsx` uploads it directly to `POST /api/media/upload` (multipart) → on success, the returned URL/size is stored in local form state → admin fills in title/description/type-specific fields → submit calls `addContent(courseId, moduleId, submoduleId, payload)` from `useCatalog.jsx`, the exact same function the Course Builder Workspace uses (Feature 3) → new `ContentEntity` row created under the chosen submodule → `refreshData()` → the new asset now also appears in the Media Library, since it's derived from the same tree.

Server-side upload handling (`MediaUploadController`): tries Cloudinary first (`cloudinaryService.uploadFile()` , `resource_type: "auto"` so images/video/PDF/PPT/DOC are all accepted); if Cloudinary isn't configured (no API credentials in `.env` /environment) or the upload throws, it **silently falls back to saving the file to local disk** under an `uploads/` folder relative to the backend's working directory, using a random UUID filename, and returns a relative `/uploads/{uuid}.{ext}` URL instead of a Cloudinary URL. `CorsConfig` also registers a static resource handler so files under `uploads/` are served back out at that same `/uploads/**` path.

7. API Details

Method	Endpoint	Request Body	Response	Auth	Notes
POST	<code>/api/media/upload</code>	multipart/form-data , field name <code>file</code>	<code>ApiResponse<{ url, name, size }></code>	None	Cloudinary if configured, else local-disk fallback; 400 if file empty; <code>spring.servlet.multipart.max-file-size=50MB</code>

There is no `GET /list`, delete, or replace endpoint for uploaded media — files are effectively write-once and only ever referenced by whatever content block's URL field points at them; an orphaned upload (uploaded but never attached to a content block, or a content block later deleted) is never cleaned up on either Cloudinary or local disk.

8. Database Flow

No dedicated table. Uploaded file URLs are stored inline wherever they're used: `CourseEntity.icon/thumbnail/bannerImage` , `CategoryEntity.logo/bannerImage/backgroundImage/thumbnail` , `ModuleEntity / SubmoduleEntity` equivalents, and `ContentEntity.videoUrl / imageUrl` /(JSON-encoded `fileUrl` inside `text` for other file types, per Feature 3 §16).

9. State Management

`mediaLibrary` is a derived/computed value inside `CatalogContext` (see §6) — there is no independent fetch or cache for it. `MediaLibrary.jsx` keeps its own local `useState` for search, type filter, sort, grid/list view, pagination, and the open preview drawer target. `UploadContentPage.jsx` keeps a large local form object plus separate upload-progress/error state for the file picker itself.

10. Component Hierarchy

- `MediaLibrary` (page)
 - `SearchBar` , `FilterDropdown` ×2 (Type, Sort By), grid/list toggle
 - grid/list of media items
 - `ContentPreviewDrawer` (slide-over preview)
- `UploadContentPage` (page)
 - cascading `Category` → `Course` → `Module` → `Submodule` selects
 - type-specific fields (`ImageUploader` , `RichNotesEditor` , plain inputs depending on selected type)

11. Sequence Diagram (Text)

Admin → `UploadContentPage.jsx` (select file) → `api.js POST /media/upload` → `MediaUploadController` → `CloudinaryService` (or local-disk fallback) → returns URL → `UploadContentPage.jsx` (fill metadata, submit) → `useCatalog.addContent()` → `api.js POST /contents` → `ContentController` → `ContentServiceImpl` → `ContentRepository.save()` → DB → `refreshData()` → `CatalogContext.mediaLibrary` recomputed → `MediaLibrary.jsx` shows the new asset

12. Dependencies

Frontend: `axios`. Backend: `cloudinary` Java SDK, Spring Web multipart support. Depends on Course Builder Workspace's content-block data model (Feature 3) since this feature has no storage of its own.

13. Validation Rules

Backend: only checks `file.isEmpty()` before attempting upload; no file-type or size validation beyond the global 50MB Spring multipart limit; no virus/content scanning. Frontend: `UploadContentPage.jsx` restricts the file picker's `accept` attribute per selected content type, which is a UI hint only — nothing stops a mismatched file from being uploaded via a direct API call.

14. Error Handling

Backend: Cloudinary exceptions are caught and logged, then the code transparently falls back to local disk — a genuine outage or misconfiguration of Cloudinary is invisible to the end user beyond a slower response, since both paths return the same `ApiResponse` shape with different URL formats. Frontend: upload failures show a toast and reset the upload-progress UI; `fileError` local state surfaces inline validation issues before a request is even sent.

15. Security

None. Any unauthenticated request can upload arbitrary files (up to 50MB, any extension) to either Cloudinary or the server's local disk — there is no allow-list of MIME types or extensions server-side.

16. Important Classes and Methods

`CloudinaryServiceImpl.init()` — `@PostConstruct` check of whether cloud name/API key/secret are all non-blank; sets an internal configured flag consulted by `MediaUploadController`.

`MediaUploadController.uploadFile()` — the Cloudinary-then-local-disk fallback chain described in §6.

`useCatalog.jsx`'s `mediaLibrary` `useMemo` — the client-side flatten-and-filter logic that makes this whole feature possible without any backend media endpoint.

17. Feature Dependencies

Course Builder Workspace (Feature 3) — Media Library has no independent existence; it is a read-only lens over content-block data owned by that feature. Course Management (Feature 2) and Category Management (Feature 1) also route their own image fields through the same `/api/media/upload` endpoint.

18. Possible Improvements

- Add the missing sidebar navigation entries for Media Library and/or Upload Content, or intentionally document them as secondary/deep-link-only screens.
- Add a real backend-tracked media table so orphaned uploads can be identified and cleaned up.
- Add server-side MIME-type/extension validation.
- Surface Cloudinary-vs-local-disk-fallback status somewhere visible (currently silent), since local-disk storage will not survive a redeploy of a stateless container.

19. Summary

Media Library has no backend of its own — it's a derived view over Course Builder content-block data. Uploads go through one shared endpoint (`/api/media/upload`) used by every image/file field across the whole app, backed by Cloudinary with a silent local-disk fallback. No auth, no file-type validation, no cleanup of orphaned files.

[↑ Back to top / index](#)

Feature 5: Learning Analytics Dashboard

1. Feature Name

Learning Analytics Dashboard (Executive Summary + 12 sub-dashboards)

2. Feature Purpose

The landing page after login (`/admin/dashboard`). A 13-tab enterprise reporting surface — Executive Summary, Learning Coverage, Learning Hours, Training Pillars, AI Transformation, Certifications, Flagship Programs, Trends, Effectiveness, Champions, Project Investment, Fresher Journey, and Future Analytics — with a shared filter bar (year/quarter/region/location/business unit/department/practice/employee grade) and a CSV export button per tab. This is the only feature in the app that touches the "HR analytics" side of the data model (employees, enrollments, certifications, fresher journeys).

3. Feature Location

Frontend

- `frontend/src/features/dashboard/Dashboard.jsx` (route `/admin/dashboard` , ~986 lines, all 13 tabs implemented in one file)

Backend

- `controller/AnalyticsController.java` → `/api/analytics/dashboard` — a single endpoint that computes and returns the entire payload for all 13 tabs in one response
- Repositories consumed directly (no service layer interposed): `EmployeeRepository` , `EnrollmentRepository` , `CertificationRepository` , `FresherJourneyRepository` , plus `CourseRepository` / `CategoryRepository` for two simple counts
- Entities: `EmployeeEntity` , `EnrollmentEntity` , `CertificationEntity` , `FresherJourneyEntity` — used **exclusively** by this feature; none of them has its own controller/DTO/mapper

4. Related Files

Frontend: `Dashboard.jsx` · `services/api.js` · `components/layout/PageHeader.jsx` · `components/ui/Badge.jsx`

Backend: `AnalyticsController.java` · `EmployeeEntity/EnrollmentEntity/CertificationEntity/FresherJourneyEntity` · their four repositories · `LMSBackendApplication.java` (seeds all the underlying data, see Feature 8)

5. Feature Entry Point

Successful login redirects to `/admin/dashboard` by default (both `App.jsx` 's root route and `LoginPage.jsx` 's post-login navigate target). `Dashboard.jsx` mounts, builds a query-string from the current (mostly `'all'` -valued) filter state, and calls `GET /api/analytics/dashboard` .

6. Complete Feature Lifecycle

On mount and on every filter change, `fetchDashboardData()` strips out any filter still set to `'all'` and sends the rest as query params → `AnalyticsController.getDashboardData()` loads *all* employees/enrollments/certifications/fresher-journeys into memory with `findAll()` (no pagination, no SQL-level filtering) → filters the in-memory employee list against each provided parameter (region/location/businessUnit/department/project/practice/employeeGrade/employeeId) using case-insensitive string equality → derives the filtered enrollment/certification/fresher-journey lists by intersecting on the surviving employee IDs → computes ~13 sections of aggregate metrics using Java streams (sums, averages, groupBy-and-count, top-N sorts) → assembles one large `Map<String, Object>` and returns it as raw JSON (not a typed DTO) → the frontend switches which section of that same response object it renders based on the active tab.

Important caveat: a large fraction of the numbers in this payload are **hardcoded literals, not derived from the seeded data at all** — e.g. `certificationGrowthPct = 14.5` , the entire `regionCoverage` / `locationCoverage` / `gradeCoverage` / `businessUnitCoverage` maps in "Learning Coverage", `topProjects` , `aiReadinessIndex = 82.4` , `aiAttendancePct = 92.5` , `aiMaturityScore = 78.5` , the entire `flagshipPrograms` list (named programs like "YMP (Young Managers)", "Quantum Shift", "Tech AI Thon" that don't correspond to any seeded course or entity), the entire `learningTrends` monthly series (Jan–May are fully hardcoded; only June partially blends in live totals), `bestRatedTrainings` / `bestRatedTrainers` (named people who don't exist in `EmployeeEntity` , e.g. "Dr. Priya Sharma"), the `skillGapAnalysis` / `suggestedCourses` / `predictiveForecasts` in "Future Analytics", and the champions' names in most non-empty-data branches. Filtering by region/department/etc. therefore has **no visible effect on most of the 13 tabs**, even though it visibly changes the handful of genuinely-computed metrics (headline employee/enrollment counts, learning hours sums, top-10 learners list).

7. API Details

Method	Endpoint	Query Params	Response	Auth	Notes
GET	/api/analytics/dashboard	year, quarter, month, region, location, businessUnit, department, project, practice, employeeGrade, employeeId (all optional)	Raw Map<String, Object> JSON, not ApiResponse -wrapped and not a typed DTO — the one exception to the ApiResponse convention used everywhere else in the backend	None	No pagination; loads full tables into memory on every call

8. Database Flow

employees (no FK — a flat dimension table: name, email, grade, region, location, business unit, department, practice, project, avatar). enrollments (FK employee_id, course_id, both NOT NULL; progress %, learning hours, status, feedback rating, recommendation score, flagship flag). certifications (FK employee_id NOT NULL; name, technology, status, completion date, AI flag). fresher_journeys (one-to-one FK employee_id NOT NULL; a milestone-date pipeline from hire → training enrollment → training completion → certification completion → project allocation → billable deployment). None of these four tables is ever written to by any controller in the running app — they are populated once, at startup, by the seeder (Feature 8) and are otherwise read-only for the lifetime of the process.

9. State Management

Local component state only — filters, activeTab, data, loading, error, all plain useState inside Dashboard.jsx. A single fetch effect re-runs whenever filters changes (tab switches do not refetch — all 13 tabs' data arrives in the one initial response and activeTab just picks which slice of data to render).

10. Component Hierarchy

Dashboard (page)
 └ filter bar (8 dropdowns)
 └ tab strip (13 tabs)
 └ per-tab metric cards / lists / simple bar-style visualizations (hand-rolled with divs + inline widths, not a charting library — recharts / chart.js etc. are not dependencies of this frontend)
 └ "Export CSV" button (client-side Blob /data-URI generation per active tab, no backend export endpoint)

11. Sequence Diagram (Text)

Admin → Dashboard.jsx (mount / change filter) → api.js GET /analytics/dashboard?... → AnalyticsController.getDashboardData() → 4× repository.findAll() → in-memory Java-stream filtering & aggregation (13 sections, several hardcoded) → single Map response → Dashboard.jsx stores whole payload → tab click re-renders from already-fetched data (no network call)

12. Dependencies

Frontend: axios, framer-motion, lucide-react. Backend: Spring Web, Spring Data JPA. Fully dependent on Database Seeding (Feature 8) for any non-zero data, since there is no other way to populate employees/enrollments/certifications/fresher journeys.

13. Validation Rules

None — every filter parameter is an untyped, unvalidated optional String / Long compared case-insensitively; an unrecognized value (typo, wrong casing handled, but a genuinely wrong value) simply filters everything out silently rather than erroring.

14. Error Handling

Frontend: a caught fetch failure sets a generic error string ("Could not connect to Spring Boot backend...") and renders an error panel; no retry button. Backend: no explicit error handling beyond whatever GlobalExceptionHandler 's catch-all provides for an unexpected exception (e.g. a malformed employeeId query param that fails Long parsing).

15. Security

None — this endpoint exposes full employee PII (names, emails, region/location/grade/department/project assignments) with no authentication or field-level redaction.

16. Important Classes and Methods

`AnalyticsController.getDashboardData()` — one very large method (~250 lines) that does filtering, aggregation, and response-shaping all inline with no delegation to a service layer; this is the single largest method in the entire backend codebase.

17. Feature Dependencies

Course Management (course titles/categories referenced in a few aggregate sections) and Database Seeding (sole data source). Does not depend on, and is not depended on by, Category/Course/Curriculum management at the write-path level — it is entirely a read-only reporting layer bolted onto the side of the LMS.

18. Possible Improvements

- Replace the large fraction of hardcoded/mock metrics with real aggregate queries once there's a genuine need for those numbers to be accurate, or clearly flag them as placeholder/demo data in the UI.
- Move filtering into the database layer (JPA Specifications / native queries) instead of loading entire tables into memory on every request — this will not scale past the current ~120-employee seed size.
- Introduce a service layer and typed response DTOs instead of one controller method building a raw `Map`.
- Restrict this endpoint once real authentication exists, given the PII it returns.

19. Summary

A single mega-endpoint, `GET /api/analytics/dashboard`, powers a 13-tab reporting UI. Roughly half of the returned metrics are genuinely computed from seeded `employees / enrollments / certifications / fresher_journeys` data; the other half are hardcoded demo values baked directly into the controller. No pagination, no service layer, no auth, and the whole feature is read-only — nothing in the running app ever writes to the four analytics entities after the initial seed.

[↑ Back to top / index](#)

Feature 6: Authentication (Mock Login)

1. Feature Name

Authentication — Login, token refresh, profile lookup

2. Feature Purpose

Gates access to the entire `/admin/*` route tree behind a login screen. Implemented end-to-end (login form → context → protected routes → Axios interceptor with refresh-on-401), but the backend behind it is a hardcoded stand-in, not real authentication — see System-Wide Notes.

3. Feature Location

Frontend: `frontend/src/pages/LoginPage.jsx` (route `/login`) · `frontend/src/hooks/useAuth.jsx` (`AuthProvider` / `useAuth`) · `frontend/src/services/authService.js` · `frontend/src/services/api.js` (interceptors) · `frontend/src/components/layout/ProtectedRoute.jsx`

Backend: `controller/AuthController.java` → `/api/auth`

4. Related Files

`App.jsx` (route wiring, `/login` outside `ProtectedRoute`, everything else inside it) · `providers.jsx` (`AuthProvider` wraps `CatalogProvider`) · `components/ui/Logo.jsx`, `Button.jsx`, `Input.jsx` (login form UI)

5. Feature Entry Point

Any unauthenticated request to an `/admin/*` route is redirected by `ProtectedRoute.jsx` to `/login` , preserving the originally-requested path in router `state` so login can redirect back to it afterward.

6. Complete Feature Lifecycle

Admin enters email/password on `LoginPage.jsx` → client-side validation (email format regex, password min length 6 — this minimum is a UI-only rule, unrelated to the actual hardcoded credentials) → `login(email, password)` in `useAuth.jsx` → `authService.login()` → `POST /api/auth/login` → `AuthController.login()` compares the (lower-cased, trimmed) email and raw password against two literal string pairs in an `if/else if` chain: `admin@xebia.com/admin123` → "Sarah Chen" (Admin role), or `instructor@xebia.com/instructor123` → "Priya Sharma" (Admin role — the "instructor" account is also granted the "Admin" role string, there is no role differentiation anywhere in the app) → on match, returns a fabricated `accessToken / refreshToken` (string-concatenated with `System.currentTimeMillis()` , not a real JWT, no signature, no expiry enforcement) plus a user object with a Dicebear-generated avatar URL → frontend stores `token / refreshToken / user` in `localStorage` under the keys `xebia-lms-token / xebia-lms-refresh-token / xebia-lms-user` → `navigate(from, { replace:true })` back to the originally-requested route (default `/admin/dashboard`).

7. API Details

Method	Endpoint	Request Body	Response	Notes
POST	<code>/api/auth/login</code>	<code>{ email, password }</code>	<code>ApiResponse<{ accessToken, refreshToken, expiresIn:3600, user:{email, fullName, role, avatar} }></code>	Only 2 hardcoded credential pairs are ever valid; 401 otherwise; 400 if either field is missing
POST	<code>/api/auth/refresh</code>	<code>{ refreshToken }</code>	<code>ApiResponse<{ accessToken, refreshToken, expiresIn }></code>	Only checks the token string starts with <code>"mock-refresh-"</code> — any string with that prefix is accepted, regardless of who issued it or when
GET	<code>/api/auth/profile</code>	—	<code>ApiResponse<{email, fullName, role, avatar}></code>	Only checks that an <code>Authorization: Bearer ...</code> header is <i>present</i> — the token value itself is never inspected; always returns the hardcoded "Sarah Chen" profile regardless of which account logged in or what the token actually is; not called anywhere in the active frontend today (dead endpoint from the frontend's perspective)

8. Database Flow

None. There is no `users` table anywhere in the schema — credentials, roles, and profile data are all literal values inside `AuthController.java` .

9. State Management

`AuthContext ( useAuth.jsx )` holds `user , token , loading , derives isAuthenticated = !!token` , and hydrates itself from `localStorage` on mount. `services/api.js` 's `Axios` response interceptor independently manages the refresh-on-401 flow and directly mutates `localStorage` (bypassing `React state`) when it needs to update the stored access token after a refresh — meaning `AuthContext.token` can go stale relative to what's actually in `localStorage` until the next full page load or explicit `login()` / `logout()` call.

10. Component Hierarchy

`LoginPage (page)` └─ `Logo , Input *2, Button`
`ProtectedRoute` wraps the entire `/admin/*` route subtree inside `App.jsx` .

11. Sequence Diagram (Text)

Admin → `LoginPage.jsx` → `useAuth.login()` → `authService.login()` → `api.js POST /auth/login` → `AuthController.login()` (hardcoded credential check) → response → `localStorage.setItem(token/refreshToken/user)` → `AuthContext` state update → `navigate(from)` → `ProtectedRoute` re-evaluates `isAuthenticated=true` → `AppLayout` renders

12. Dependencies

Frontend: `axios`, `react-router-dom`, `framer-motion`. Backend: none beyond Spring Web — no Spring Security, no password hashing library, no JWT library.

13. Validation Rules

Frontend: email regex + 6-char password minimum, purely cosmetic given the fixed real credentials. Backend: only null-checks on `email` / `password` presence; no format validation, no rate limiting/lockout after repeated failures.

14. Error Handling

Frontend distinguishes three failure shapes in both `LoginPage.jsx` and `useAuth.jsx`: a server response (uses its message, or a generic 401/403/"Invalid email or password" fallback), a request that was sent but got no response (network/CORS message naming the configured API URL), or any other thrown error. Backend returns 400 for missing fields, 401 for a credential mismatch — no other error paths exist in `AuthController`.

15. Security

Effectively none — see System-Wide Notes. Credentials are plaintext literals in source control, passwords are compared with `.equals()` rather than hashed, tokens are unsigned, and no backend endpoint (including the ones documented in every other Feature in this document) ever checks the `Authorization` header at all.

16. Important Classes and Methods

`AuthController.login/refresh/getProfile/createAuthResponse()` — the entire "auth system" in one 90-line file.
`api.js`'s response interceptor — implements a real-looking refresh-and-retry pattern (marks `originalRequest._retry`, swaps in the new token, replays the original call) purely for a backend that will accept any `"mock-refresh-"`-prefixed string.

17. Feature Dependencies

Gates every other feature in this document at the frontend routing layer only — none of the other controllers actually depend on or check anything this feature produces.

18. Possible Improvements

- This is the highest-priority gap in the whole codebase for any real deployment: replace with Spring Security + a real user store (hashed passwords), issue signed JWTs, and validate the `Authorization` header on every controller (or centrally via a filter/interceptor).
- Add role-based authorization once real roles exist — today "Admin" and "Instructor" logins are functionally identical.
- Add login rate-limiting/lockout.

19. Summary

A fully-built-out login UI and token-refresh plumbing sitting in front of a backend that hardcodes two credential pairs and issues fake, unvalidated tokens. No endpoint in the app — including this one's own `/profile` endpoint — actually enforces the token it issues.

[↑ Back to top / index](#)

Feature 7: Application Shell — Layout, Routing, Toasts & Global State

1. Feature Name

Application Shell (Sidebar navigation, route table, toast notifications, catalog hydration)

2. Feature Purpose

The chrome every other feature lives inside: the fixed purple sidebar visible in the reference screenshot (Dashboard / Categories / Courses, teal active-state pill, user footer with avatar + light/dark theme toggle + logout), the top-level React Router route table, and the toast notification system used by every mutation in the app.

3. Feature Location

Frontend: `frontend/src/App.jsx` (route table) · `frontend/src/providers.jsx` (provider nesting) · `frontend/src/components/layout/AppLayout.jsx` · `frontend/src/components/layout/Sidebar.jsx` · `frontend/src/hooks/useToast.jsx` · `frontend/src/hooks/useCatalog.jsx` (app-wide data hydration on mount)

4. Related Files

`components/layout/Breadcrumb.jsx` (used by a few deep pages) · `components/ui/Logo.jsx` · `constants/index.js` (`BRAND_DEFAULTS`) · `index.css` (Tailwind theme tokens, CSS variables for `--brand-primary` / `--brand-secondary` set at runtime by `AppLayout.jsx`)
 `frontend/src/components/layout/Header.jsx` (512 lines) is completely unused — grepping the entire active `src/` tree finds no import of it anywhere; `AppLayout.jsx` renders only `Sidebar` . It appears to be a fully-built alternative top-bar (branding/notifications/search, based on its size) that was superseded by folding the user menu into the `Sidebar`'s footer instead, but was never deleted.

5. Feature Entry Point

`main.jsx` renders `<App/>` → `App.jsx` wraps everything in `BrowserRouter` then `Providers` (`QueryClientProvider` → `ToastProvider` → `AuthProvider` → `CatalogProvider`) → route table splits into `/login` (public) and `/*` (wrapped in `ProtectedRoute` then `AppLayout`).

6. Complete Feature Lifecycle

Hydration: as soon as `CatalogProvider` mounts (which only happens once `AuthProvider` has resolved, since it's nested inside it — though notably `CatalogProvider`'s own `refreshData()` effect does not actually check `isAuthenticated` before firing, so it will attempt to call `/categories` and `/courses` even on the public `/login` route, silently failing/retrying since `ProtectedRoute` hasn't blocked anything at the API layer — only at the route layer), it fires `refreshData()` once, then loads branding from `localStorage` and a small set of hardcoded default notifications, then flips `hydrated=true` ; every page checks this flag and renders nothing until it's true, avoiding a flash of empty state.

Navigation: `Sidebar.jsx`'s `NAV_ITEMS` is a 3-item hardcoded array (Dashboard, Categories, Courses) rendered as `react-router-dom` `Link` s; active-state is computed by prefix-matching the current pathname (with a special case so `/admin/courses/*` and `/admin/curriculum/*` both highlight "Courses").

Theme toggle: reads/writes a `theme` key in `localStorage` and toggles a `dark` class on `document.documentElement` ; independent of the branding primary/secondary color CSS variables set by `AppLayout.jsx` .

Toasts: `ToastProvider` (`useToast.jsx`) exposes `showToast(message, type)` , used by virtually every mutation across every feature in this document to report success/failure.

7. API Details

No dedicated endpoints of its own — indirectly triggers `GET /api/categories` + `GET /api/courses` on every app load via `CatalogProvider`'s hydration effect (see §6 caveat about this firing even pre-login).

8. Database Flow

N/A — pure frontend shell feature.

9. State Management

Three nested Contexts carry app-wide concerns: `ToastContext` (queue of active toasts, auto-dismiss timers), `AuthContext` (Feature 6), `CatalogContext` (Features 1–4's shared data store). `QueryClientProvider` is present but, as noted in Feature 1 §9, not actually used by the live data-fetching code path today.

10. Component Hierarchy

`App`
└ Router (BrowserRouter)
└ Providers (QueryClientProvider → ToastProvider → AuthProvider → CatalogProvider)


```
└─ Routes
  └─ /login → LoginPage
└─ /* → ProtectedRoute → AppLayout ( Sidebar + <main> ) → nested Routes for every admin page
```

11. Sequence Diagram (Text)

Browser loads / → main.jsx → App.jsx → Providers mount (Toast → Auth (hydrates from localStorage) → Catalog (fires refreshData regardless of auth state)) → Router matches route → ProtectedRoute checks isAuthenticated → AppLayout renders Sidebar + routed page

12. Dependencies

react-router-dom, @tanstack/react-query (present but underused), framer-motion, lucide-react, clsx / tailwind-merge (via the cn() utility in utils/index.js).

13. Validation Rules

N/A.

14. Error Handling

CatalogProvider.refreshData() catches any fetch failure and shows a toast ("Error syncing with backend server") but does not set any error-boundary state — a fully-failed initial load leaves categories / courses as empty arrays with hydrated still eventually set to true, so pages render their normal "no data" empty states rather than a dedicated connection-error screen (contrast with the Dashboard, Feature 5, which does show a dedicated error panel).

15. Security

ProtectedRoute is the entire security boundary for the frontend, and as covered in Feature 6, it is not backed by any real server-side enforcement — it only prevents the React Router tree from rendering, not API access.

16. Important Classes and Methods

AppLayout.jsx's useEffect — writes branding.primaryColor / secondaryColor to CSS custom properties on document.documentElement, letting the whole Tailwind theme (via brand-primary / brand-secondary utility classes defined in tailwind.config.js) be re-themed at runtime without a rebuild — though no UI in the live app actually lets an admin change branding today (a "Branding Settings" screen is documented in the dead Next.js tree's README but does not exist under src/).

17. Feature Dependencies

Every other feature in this document renders inside this shell and depends on its three Contexts.

18. Possible Improvements

- Gate CatalogProvider's hydration effect behind isAuthenticated to avoid a doomed API call on the public login route.
- Delete the unused Header.jsx, or build the branding-settings screen its presence (and the dead tree's README) implies was planned.
- Add a dedicated connection-error state to the shell so every page doesn't have to reimplement its own "backend unreachable" handling.

19. Summary

A conventional React Router + Context Provider shell. Sidebar navigation matches the reference screenshot exactly (3 nav items, teal active state, purple #4a1e47 background, user footer with theme toggle and logout). A large, fully-built alternate header component exists on disk but is dead code. Catalog data hydrates once per app load and is shared by every feature via Context rather than a per-page fetch.

[↑ Back to top / index](#)

Feature 8: Database Seeding

1. Feature Name

Database Seeding (startup `CommandLineRunner`)

2. Feature Purpose

Populates a fresh (or wiped) database with the demo data visible in the reference screenshot — 2 categories ("Cloud Computing", "Artificial Intelligence"), 2 courses ("AWS Cloud Practitioner & DevOps Essentials", "GenAI & Prompt Engineering Foundation"), 1 module, 1 submodule, 1 content item — plus a much larger synthetic HR dataset (120 employees, their enrollments, certifications, and 30 fresher journeys) that exists purely to feed the Analytics Dashboard (Feature 5).

3. Feature Location

`backend/src/main/java/com/geeknito/LMS_backend/LMSBackendApplication.java` — the `@Bean public CommandLineRunner databaseCleanser(...)` method, defined inline in the same file as the `main()` method (there is no separate `DatabaseSeeder` class).

4. Related Files

Every repository in the app is injected into this one bean: `CategoryRepository` , `CourseRepository` , `ModuleRepository` , `SubmoduleRepository` , `ContentRepository` , `EmployeeRepository` , `EnrollmentRepository` , `CertificationRepository` , `FresherJourneyRepository` .

5. Feature Entry Point

Runs automatically on every backend process start, before the application finishes booting — `CommandLineRunner` beans execute once Spring's context is fully initialized but before the embedded server starts accepting external traffic in earnest.

6. Complete Feature Lifecycle

Step 1 — Wipe: deletes all rows from `fresher_journeys` , `certifications` , `enrollments` , `employees` , then `contents` , `submodules` , `modules` , `courses` , `categories` — in that specific dependency order, using ordinary `repository.deleteAll()` calls (not a raw `TRUNCATE`) so standard JPA cascade/FK rules apply.

Step 2 — Reseed core catalog: builds 2 `CategoryEntity` rows, 2 `CourseEntity` rows (one per category, using a fixed Cloudinary-hosted placeholder image URL for every icon/thumbnail/banner field on both courses and both categories), 1 `ModuleEntity` under the AWS course, 1 `SubmoduleEntity` under that module, and 1 `ContentEntity` (a Markdown "notes" block) under that submodule. This exactly matches what the reference screenshot shows: 2 courses, one with "1 Modules · 1 Submodules" and one with "0 Modules · 0 Submodules".

Step 3 — Reseed HR/analytics data: generates 120 `EmployeeEntity` rows from small hardcoded name/region/location/business-unit/department/practice-array pools using a **fixed random seed** (`new Random(42)`), so the generated dataset is deterministic across restarts. For each employee: a ~75% chance of an AWS-course enrollment and a ~50% chance of a GenAI-course enrollment (each with random progress/hours/status/feedback/recommendation-score/flagship-flag), and 0–2 random certifications drawn from a 7-item name pool with randomly-assigned status and an AI-detection heuristic (name contains "AI"/"Kiro"/"Copilot", or a coin flip). The first 30 employees generated are then *also* force-set to grade "E1" and given a synthetic `FresherJourneyEntity` with a plausible hire → training → certification → allocation → deployment date pipeline.

7. API Details

N/A — not exposed via any HTTP endpoint; runs as part of process startup only.

8. Database Flow

Direct repository writes across all 9 tables in the schema, entirely inside the implicit transaction Spring wraps a `CommandLineRunner` bean's single method in — a failure partway through rolls back the whole seed.

9. State Management

N/A (backend startup only).

10. Component Hierarchy

N/A.

11. Sequence Diagram (Text)

Backend process starts → Spring context initializes → `databaseCleanser` `CommandLineRunner` executes → `deleteAll()` ×9 tables (dependency order) → save 2 categories → save 2 courses → save 1 module → save 1 submodule → save 1 content → save 120 employees → save ~90–180 enrollments (random per employee) → save ~0–240 certifications (random per employee) → save 30 fresher journeys → application finishes booting and starts serving traffic

12. Dependencies

All 9 JPA repositories in the app; no external dependencies.

13. Validation Rules

None — entities are built directly via their Lombok `@Builder`, bypassing every `@Valid` /DTO validation path that applies to normal HTTP-driven writes.

14. Error Handling

None beyond Spring's default behavior of failing application startup if the `CommandLineRunner` throws (the whole seed is one implicit transaction, so a mid-seed failure rolls back cleanly rather than leaving partial data — but the backend process will not come up at all).

15. Security

This is a destructive operation that runs **unconditionally on every single restart**, with no environment/profile guard and no "seed only if empty" check — every backend restart wipes and regenerates *all* data in every table, including anything an admin created through the UI since the last restart. This is the single highest-risk piece of code in the repository if it were ever pointed at a real deployment: there is no way to preserve admin-entered categories/courses across a redeploy without first removing or gating this bean.

16. Important Classes and Methods

`LMSBackendApplication.databaseCleanser(...)` — the entire seeding routine, in one large lambda passed to `CommandLineRunner`.
`LMSBackendApplication.main()` also contains a hand-rolled `.env` file parser (reads key=value lines, strips quotes, calls `System.setProperty()`) used specifically to get Clouinary credentials into Spring's property resolution before `SpringApplication.run()` is invoked — this exists because `.env` is not a Spring-native config format.

17. Feature Dependencies

Touches every entity in the data model; has no feature dependencies of its own. Feature 5 (Analytics Dashboard) has no other data source and is entirely dependent on this seeder having run.

18. Possible Improvements

- Gate the whole bean behind a Spring profile (e.g. only run when `spring.profiles.active=dev`) or an explicit config flag before this codebase is ever pointed at a persistent/shared database — this is the top-priority fix for anyone planning a real deployment.
- Replace the unconditional wipe-and-reseed with idempotent "seed only if the table is empty" logic.
- Move the seed data out of hardcoded Java into external fixtures (SQL/JSON/CSV) for easier maintenance.

19. Summary

Runs automatically on every backend startup. Deletes and regenerates all 9 tables — a small, deterministic catalog (2 categories/2 courses/1 module/1 submodule/1 content, matching the reference screenshot exactly) plus a much larger randomized-but-seeded (120 employees, their enrollments/certifications, 30 fresher journeys) HR dataset used exclusively by the Analytics Dashboard. No environment guard exists — this must be disabled or gated before any production-like use.

Feature 9: Cross-Cutting Infrastructure

These aren't user-facing "features" but are foundational and touched by every feature above.

9.1 Centralized Axios Client (`services/api.js`)

Single Axios instance, `baseUrl` from `VITE_API_URL` env var (default `/api`, proxied to `http://localhost:8082` in dev by `vite.config.js`). Request interceptor injects `Authorization: Bearer <token>` from `localStorage.getItem('xebia-lms-token')` if present. Response interceptor implements a real 401-triggered refresh-and-retry flow (see Feature 6) — but again, no backend endpoint other than `/auth/*` itself ever inspects this header, so the interceptor is fully functional plumbing wired to a backend that doesn't need it yet.

9.2 Global Exception Handling (`GlobalExceptionHandler.java`)

`@ControllerAdvice` mapping `ResourceNotFoundException` → 404, `MethodArgumentNotValidException` → 400 (field-name → message map), `IllegalArgumentException` → 400, and a catch-all `Exception` → 500 with the message echoed back and the full stack trace printed to the console — used uniformly by every controller via the shared `ApiResponse` wrapper (`{ message, data, timestamp }`), with the sole exception of `AnalyticsController` (Feature 5), which returns a raw unwrapped `Map` instead of an `ApiResponse`.

9.3 CORS Configuration (`config/CorsConfig.java`)

One global `WebMvcConfigurer` bean: `/**` is opened to any `http://localhost:*` or `http://127.0.0.1:*` origin, methods `GET/POST/PUT/DELETE/OPTIONS`, all headers, credentials allowed. No controller in the codebase carries a per-endpoint `@CrossOrigin` override — this single bean is the entire CORS story. The same class also registers the static resource handler that serves locally-uploaded files back out under `/uploads/**` (Feature 4's Cloudinary fallback path).

9.4 The Two Parallel Frontend Trees

As covered in the System-Wide Notes, `frontend/app`, `components`, `features`, `hooks`, `services`, `constants`, `utils`, and `query` (all siblings of `frontend/src`, at the frontend project root) are an inert, unbuildable Next.js 15 App Router prototype — real files with real (sometimes quite different) implementations of category/course/content management, but not reachable by any script in `package.json` and not depended on by `next` (which isn't installed). Anyone editing this codebase should treat everything outside `frontend/src/` as historical reference material only, not as a second live surface.

9.5 Dead Components Inside the Active Tree

Beyond the parallel Next.js tree, the *active* `frontend/src/` tree itself also contains dead code that a new contributor could easily mistake for live:

- `components/layout/Header.jsx` (512 lines, unused — Feature 7)
- `features/module/ModuleManager.jsx` and `features/content/ContentManager.jsx` (unused alternative CRUD screens — Feature 3)
- `components/builder/CourseHierarchyTree.jsx`, `HierarchyContextMenu.jsx`, `CourseDashboardCards.jsx`, `CoursePublishDialog.jsx` (unused sibling components to the monolithic `CourseBuilderWorkspace.jsx` — Feature 3)
- `features/curriculum/CurriculumLanding.jsx` is technically routed (`/admin/curriculum`) but unreachable through any in-app link — it only renders if a user manually navigates to that exact URL without a course ID.

9.6 Manual (Non-MapStruct) Mapping Convention

Despite living in packages named `mapper`, none of `CategoryMapper`, `CourseMapper`, `ModuleMapper`, `SubmoduleMapper`, or `ContentMapper` uses `MapStruct` (no `@Mapper` annotation, no generated implementation) — `pom.xml` does not include the `MapStruct` dependency at all. Every one of these classes is a plain collection of static methods (`toEntity`, `toResponseDTO`, `updateEntity`) hand-written per entity, consistently structured across all five.

9.7 Hard-Delete-Everywhere Convention

Every `*ServiceImpl.delete()` method in the codebase (Category, Course, Module, Submodule, Content) calls the repository's `delete(entity)` directly — there is no soft-delete convention anywhere on the backend, despite every primary entity carrying an `isActive` boolean that *looks* like it exists for that purpose (and despite the Category Management UI's "Deleted"/"Restore" affordances implying one exists — see Feature 1). The `isActive` flag is set once at creation (defaulting `true`) and, on the entities that expose it via their update DTOs, can be toggled by an explicit edit — but never by a delete action.

9.8 Media Upload's Dual-Path Storage

Covered in depth in Feature 4: `MediaUploadController` tries Cloudinary first, silently falls back to local disk under an `uploads/` folder relative to wherever the backend process's working directory happens to be — meaning uploaded files' durability depends entirely on whether Cloudinary credentials are present in the deployment environment, with no visible indicator to the admin of which path was actually used for a given file.

Appendix A: Backend REST Endpoint Reference

Resource	Method	Path	Used by Active Frontend?
Category	GET	/api/categories	✓
Category	GET	/api/categories/{id}	✓ (edit form prefill)
Category	GET	/api/categories/{categoryId}/courses	✗
Category	POST	/api/categories	✓
Category	PUT	/api/categories/{id}	✓
Category	DELETE	/api/categories/{id}	✓ (hard delete, cascades)
Course	GET	/api/courses	✓
Course	GET	/api/courses/{id}	✓ (edit form + builder workspace)
Course	POST	/api/courses	✓
Course	PUT	/api/courses/{id}	✓
Course	DELETE	/api/courses/{id}	✓ (hard delete, cascades)
Module	GET	/api/modules	✗ (flat list, unused)
Module	GET	/api/modules/{id}	✗ (not called; workspace reads modules off the course tree)
Module	POST / PUT / DELETE	/api/modules / /api/modules/{id}	✓
Submodule	GET	/api/submodules , /api/submodules/{id}	✗
Submodule	POST / PUT / DELETE	/api/submodules / /api/submodules/{id}	✓
Content	GET	/api/contents , /api/contents/{id}	✗
Content	POST / PUT / DELETE	/api/contents / /api/contents/{id}	✓
Media	POST	/api/media/upload	✓ (Cloudinary, falls back to local disk)
Analytics	GET	/api/analytics/dashboard	✓
Auth	POST	/api/auth/login	✓
Auth	POST	/api/auth/refresh	✓ (via Axios interceptor)
Auth	GET	/api/auth/profile	✗

Appendix B: Entity-Relationship Summary

CategoryEntity (1) —< CourseEntity (N)	[category_id FK, NOT NULL, cascade ALL]
CourseEntity (1) —< ModuleEntity (N)	[course_id FK, NOT NULL, cascade ALL]
ModuleEntity (1) —< SubmoduleEntity (N)	[module_id FK, NOT NULL, cascade ALL; slug is globally UNIQUE]
SubmoduleEntity(1) —< ContentEntity (N)	[submodule_id FK, NOT NULL, cascade ALL]
EmployeeEntity (1) —< EnrollmentEntity (N)	[employee_id FK, NOT NULL]
CourseEntity (1) —< EnrollmentEntity (N)	[course_id FK, NOT NULL]
EmployeeEntity (1) —< CertificationEntity (N)	[employee_id FK, NOT NULL]
EmployeeEntity (1) —1 FresherJourneyEntity	[employee_id FK, NOT NULL, one-to-one]

All catalog entities (`CategoryEntity` , `CourseEntity` , `ModuleEntity` , `SubmoduleEntity` , `ContentEntity`) use Lombok `@Data` `@Builder` `@NoArgsConstructor` `@AllArgsConstructor` , Hibernate `@CreationTimestamp` / `@UpdateTimestamp` for `created_at` / `updated_at` , and `IDENTITY` primary key generation. The HR/analytics entities (`EmployeeEntity` , `EnrollmentEntity` , `CertificationEntity` , `FresherJourneyEntity`) follow the same Lombok conventions but have no `isActive` flag and no update timestamp — they are treated as write-once, seed-time-only data.

[↑ Back to top / index](#)